

Stencils 在解决诸如流体动力学、热传导、燃烧、天气预报、气候模拟和电磁学等应用领域的偏微分方程数值方法中起着基础性作用。Stencil 算法处理的数据是具有物理意义的离散化数量，如质量、速度、力、加速度、温度、电场和能量，它们之间的关系受到微分方程的控制。Stencil 的常见用途是根据输入变量值范围内的函数值来近似函数的导数值。Stencil 与卷积有很强的相似性，因为它们都是根据多维数组中当前位置的元素值以及另一个多维数组中邻域内的元素值来计算多维数组中元素的新值。因此，stencils 也需要处理边缘单元和虚拟单元。与卷积不同，stencil 计算用于迭代地解决感兴趣域内的连续可微函数的值。用于 stencil 邻域中元素的数据元素和权重系数受到正在解决的微分方程的控制。一些 stencil 模式适合于优化，而这些优化在卷积中不适用。在通过域迭代地传播初始条件的求解器中，输出值的计算可能存在依赖关系，并且需要按照某些顺序约束进行。此外，由于解决微分问题时的数值精度要求，stencils 处理的数据往往是高精度浮点数据，这会消耗更多芯片内存用于平铺技术。由于这些差异，stencils 倾向于激发不同于卷积的优化方法。

Background

在使用计算机对函数、模型、变量和方程进行数值评估和求解的过程中，第一步是将它们转换为离散表示形式。例如，图 8.1A 展示了正弦函数 $y = \sin(x)$ 在 $0 \leq x \leq \pi$ 范围内的图像。图 8.1B 展示了一个一维 (1D) 正则 (结构化) 网格的设计，其中的七个网格点对应于等间距 ($\pi/6$) 分布的 x 值。一般来说，结构化网格覆盖了一个 n 维欧几里得空间，其中的平行四边形相同 (例如，一维中是线段，二维中是矩形，三维中是立方体)。正如后面我们将看到的，对于结构化网格，变量的导数可以方便地表示为有限差分。因此，结构化网格主要用于有限差分方法。非结构化网格更加复杂，通常用于有限元和有限体积方法。为了简化起见，本书只使用规则网格和因此有限差分方法。

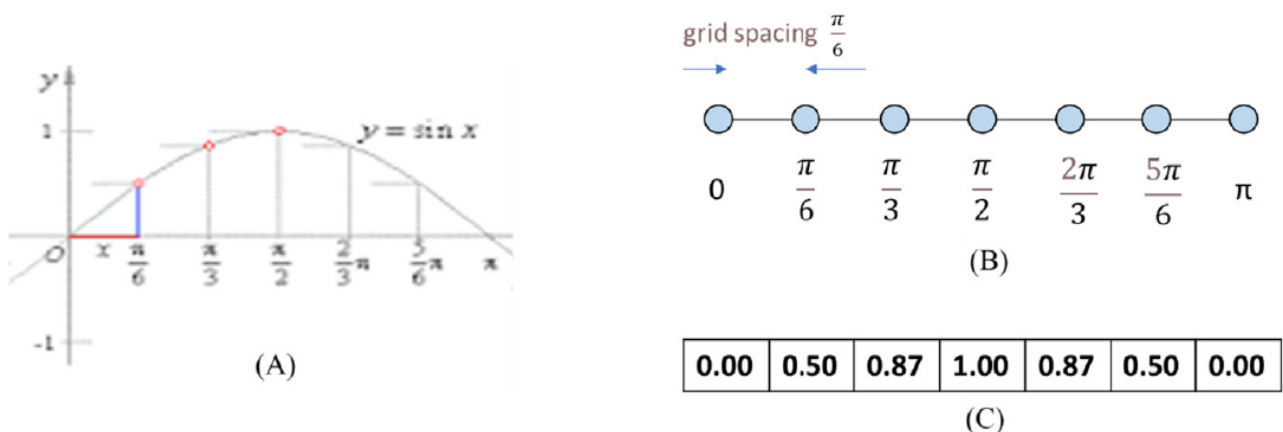


图8.1 (A) 正弦函数作为在 $0 \leq x \leq \pi$ 范围内连续可微的函数。(B) 设计了一个具有常间距 ($\pi/6$) 的正则网格，用于离散化网格点之间的距离。(C) 正弦函数在 $0 \leq x \leq \pi$ 范围内的离散表示结果。

图 8.1C 展示了结果的离散表示形式，其中正弦函数在七个网格点处的值被表示出来。在这种情况下，表示形式存储在一维数组 F 中。注意， x 值被隐式假定为 $i * \pi/6$ ，其中 i 是数组元素的索引。例如，对应于元素 $F[2]$ 的 x 值为 0.87 ，这是 $2 * \pi/6$ 对应的正弦值。

在离散表示中，需要使用线性或样条插值等插值技术来推导出在不对应任何网格点的 x 值处函数的近似值。表示形式的保真度或这些近似插值技术得到的函数值的准确程度取决于网格点之间的间距：间距越小，近似值越准确。通过减小间距，可以提高表示的准确性，但会增加存储成本，以及我们将在解决偏微分方程时看到的计算量。

离散表示的准确性也取决于所使用数字的精度。由于我们在逼近连续函数，因此通常使用浮点数表示网格点的值。目前，主流的 CPU 和 GPU 支持双精度 (64 位)、单精度 (32 位) 和半精度 (16 位) 的表示。在这三种表示中，双精度数提供了最佳的精度和最高的准确性。然而，现代 CPU 和 GPU 通常对于单精度和半精度的算

术运算具有更高的计算吞吐量。此外，由于双精度数包含更多位，读写双精度数会消耗更多的内存带宽。存储这些双精度数也需要更多的内存容量。这对于需要在芯片内存和寄存器中存储大量网格点值的瓦片技术构成了重大挑战。

让我们稍微正式一些地讨论一下“Stencil”的定义。在数学中，“Stencil”是应用在结构化网格的每个点上的一种几何权重的几何图案。该模式指定了如何通过使用数值逼近算法从相邻点的值来推导出感兴趣的网格点的值。例如，一个Stencil可以指定如何通过使用点及其邻居点的函数值之间的有限差分来逼近感兴趣点的函数的导数值。由于偏微分方程表达了函数、变量及其导数之间的关系，因此“Stencil”为指定有限差分方法如何数值计算偏微分方程的解提供了便利的基础。

例如，假设我们将 $f(x)$ 离散化为一个一维网格数组 F ，我们想要计算 $f(x)$ 的离散导数 $f'(x)$ 。我们可以使用经典的有限差分逼近方法来计算一阶导数：

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} + O(h^2)$$

也就是说，函数在点 x 处的导数可以通过两个相邻点的函数值的差除以这些相邻点的 x 值的差来近似计算。值 h 是网格中相邻点之间的间距。误差用术语 $O(h^2)$ 表示，意味着误差与 h 的平方成正比。显然， h 值越小，近似就越好。在我们的示例中，图 8.1 中的 h 值为 0.52 或 $\pi/6$ 。这个值并不足够小以使得近似误差可以忽略不计，但应该能够得到一个相当接近的近似值。

由于网格间距是 h ，当前估计的 $f(x+h)$ 、 $f(x)$ 和 $f(x-h)$ 值分别存储在数组 $F[i+1]$ 、 $F[i]$ 和 $F[i-1]$ 中，其中 $x = i * h$ 。因此，我们可以计算出 $f(x)$ 在每个网格点处的导数值并存储到一个输出数组 FD 中：

$$FD[i] = \frac{F[i+1] - F[i-1]}{2 * h}$$

这个表达式可以重写为。

$$FD[i] = \frac{-1}{2 * h} * F[i-1] + \frac{1}{2 * h} * F[i+1]$$

也就是说，计算网格点处估计函数导数值的计算涉及到网格点 $[i+1, i, i-1]$ 处的当前估计函数值，并且使用系数 $[-\frac{1}{2h}, 0, \frac{1}{2h}]$ ，这定义了一个一维三点 stencil，如图 8.2A 所示。如果我们要近似网格点处更高阶的导数值，我们需要使用更高阶的有限差分。例如，如果微分方程包含 $f(x)$ 的二阶导数，我们将使用涉及到 $[i+2, i+1, i, i-1, i-2]$ 的 stencil，这是一个一维五点 stencil，如图 8.2B 所示。一般来说，如果方程涉及到 $f(x)$ 的直到第 n 阶导数，stencil 将涉及到中心网格点两侧的 n 个网格点。图 8.2C 展示了一个一维七点 stencil。中心点两侧的网格点数量称为 stencil 的阶数，因为它反映了近似的导数阶数。根据这个定义，图 8.3 中的 stencil 分别是阶数 1、2 和 3。

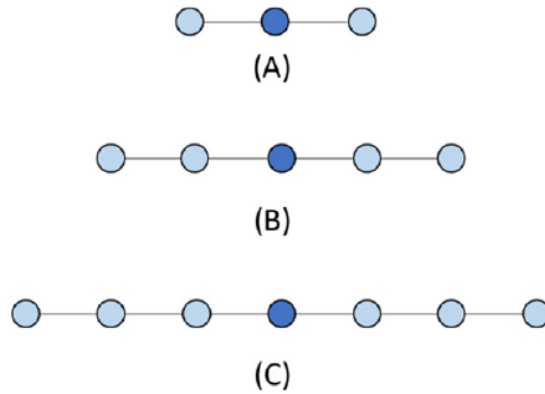


图8.2 一维 **stencil** 示例。 (A) 三点 (阶数 1) **stencil**。 (B) 五点 (阶数 2) **stencil**。 (C) 七点 (阶数 3) **stencil**。

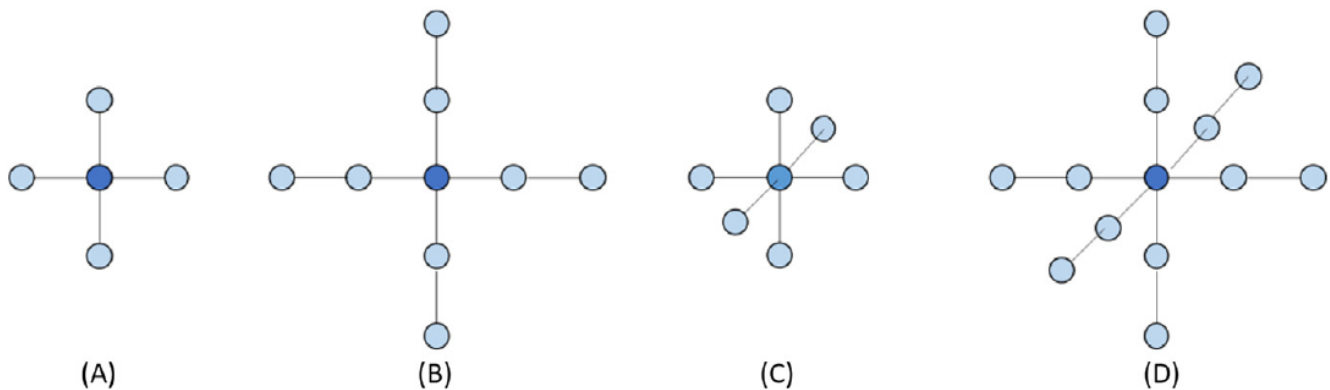


图8.3 (A) 二维五点 **stencil** (阶数 1)。 (B) 二维九点 **stencil** (阶数 2)。 (C) 三维七点 **stencil** (阶数 1)。 (D) 三维十三点 **stencil** (阶数 2)。

显而易见，解决一个涉及两个变量的偏微分方程需要将函数值离散化成二维网格，并且我们将使用二维 **stencil** 来计算近似的偏导数。如果偏微分方程仅涉及到其中一个变量的偏导数，例如 $\frac{\partial f(x,y)}{\partial x}$, $\frac{\partial f(x,y)}{\partial y}$ ，但不涉及 $\frac{\partial f(x,y)}{\partial x \partial y}$ ，我们可以使用二维 **stencil**，其中所选的网格点沿着 x 轴和 y 轴。例如，对于一个仅涉及到 x 和 y 的一阶偏导数的偏微分方程，我们可以使用一个二维 **stencil**，其中包括沿着 x 轴和 y 轴中心点两侧的两个网格点，这导致了图 8.3A 中的二维五点 **stencil**。如果方程涉及到 x 或 y 的二阶导数，我们将使用一个二维九点 **stencil**，如图 8.3B 所示。

图 8.4 总结了离散化、数值网格和 **stencil** 在网格点上的应用的概念。函数被离散化为它们的网格点值，这些值存储在多维数组中。在图 8.4 中，一个涉及两个变量的函数被离散化为一个二维网格，存储为一个二维数组。图 8.4 中使用的 **stencil** 是二维的，用于计算每个网格点的近似导数值（输出），其值来自于相邻网格点和网格点本身的函数值。在本章中，我们将专注于 **stencil** 应用于所有相关的输入网格点以生成所有网格点的输出值的计算模式，这将被称为 **stencil** 扫描。

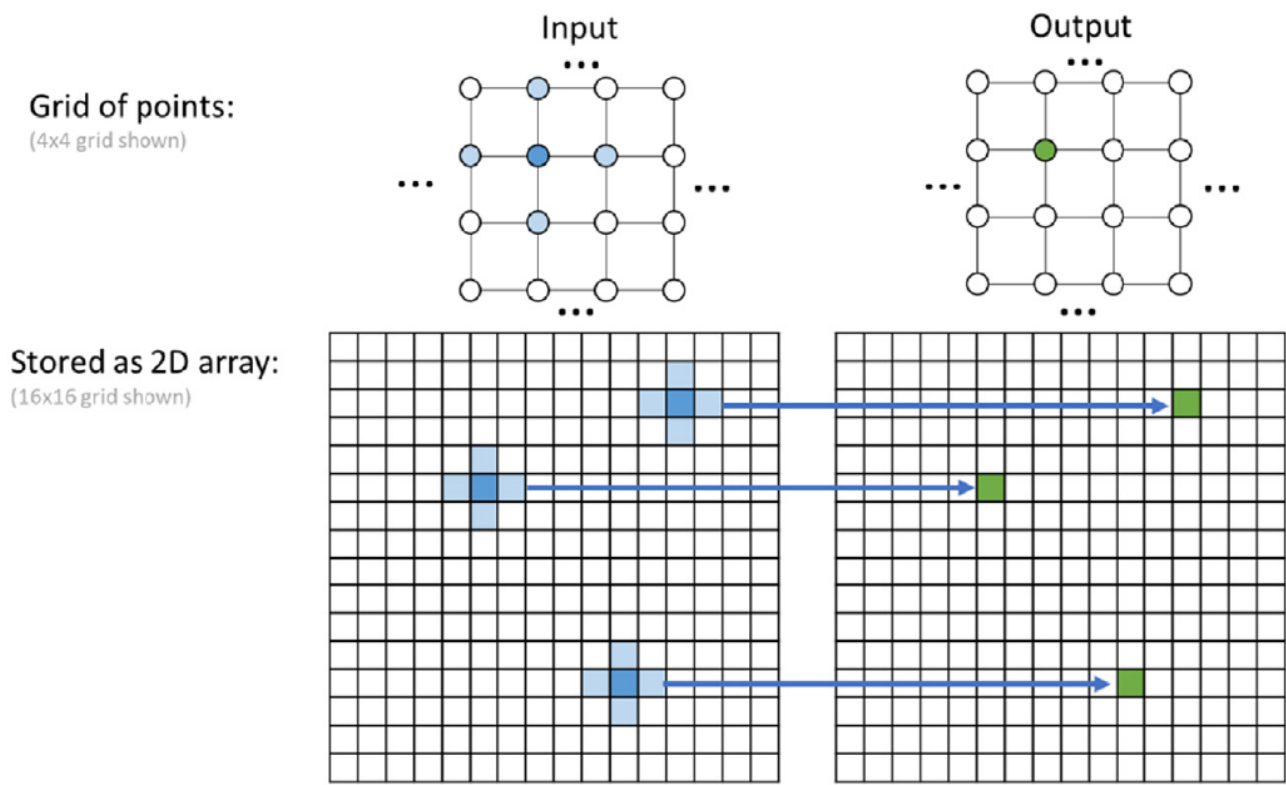


图8.4 一个二维网格的例子，以及用于计算网格点上近似导数值五点（一阶）stencil。

8.2 并行stencil：基本算法

我们首先介绍一个基本的stencil扫描内核。为简单起见，我们假设在生成stencil扫描内部输出网格点值时，输出网格点之间没有依赖关系。我们进一步假设边界上的网格点值存储边界条件，并且不会从输入到输出发生变化，如图8.5所示。也就是说，输出网格中阴影内部区域将被计算，而未阴影的边界单元格将保持与输入值相同。这是一个合理的假设，因为stencil主要用于解决带有边界条件的微分方程。

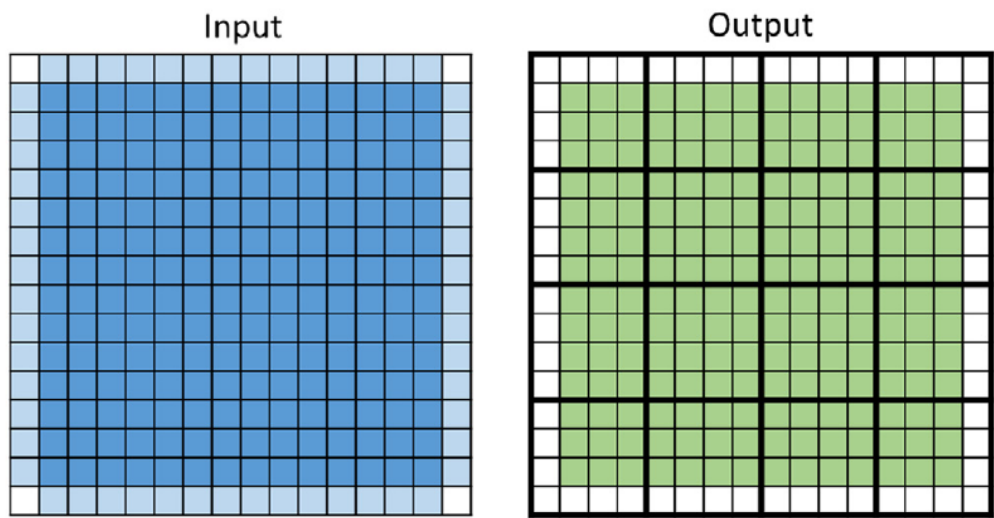


图8.5 简化边界条件。边界单元格包含的边界条件在每次迭代中都不会更新。因此，在每次stencil扫描中只需计算内部输出网格点的值。

```

01  __global__ void stencil_kernel(float* in, float* out, unsigned int N) {
02      unsigned int i = blockIdx.z*blockDim.z + threadIdx.z;
03      unsigned int j = blockIdx.y*blockDim.y + threadIdx.y;
04      unsigned int k = blockIdx.x*blockDim.x + threadIdx.x;
05      if(i >= 1 && i < N - 1 && j >= 1 && j < N - 1 && k >= 1 && k < N - 1) {
06          out[i*N*N + j*N + k] = c0*in[i*N*N + j*N + k]
07                                  + c1*in[i*N*N + j*N + (k - 1)]
08                                  + c2*in[i*N*N + j*N + (k + 1)]
09                                  + c3*in[i*N*N + (j - 1)*N + k]
10                                  + c4*in[i*N*N + (j + 1)*N + k]
11                                  + c5*in[(i - 1)*N*N + j*N + k]
12                                  + c6*in[(i + 1)*N*N + j*N + k];
13      }
14  }

```

图8.6 一个基本的stencil扫描kernel。

图8.6显示了一个执行stencil扫描的基本内核。该内核假设每个线程块负责计算输出网格值的一个瓦片，并且每个线程分配给一个输出网格点。图8.5中显示了一个输出网格的2D切片示例，其中每个线程块负责一个4x3x4的输出瓦片。然而，由于大多数现实世界的应用程序解决三维（3D）微分方程，图8.6中的内核假设一个3D网格和一个三维七点stencil，类似于图8.3C中的stencil。线程分配到网格点的方式是使用涉及blockIdx、blockDim和threadIdx的x、y和z字段的熟悉的线性表达式（第02-04行）。一旦每个线程被分配到一个3D网格点，就会对该网格点和所有相邻网格点的输入值乘以不同的系数（第06-12行的c0到c6）并进行相加。这些系数的值取决于正在解决的微分方程，正如我们在背景部分中所解释的那样。

现在让我们计算图8.6中内核的浮点数到全局内存访问比例。每个线程执行13次浮点运算（七次乘法和六次加法），并加载7个每个4字节的输入值。因此，该内核的浮点数到全局内存访问比率为 $13/(7*4)$ 约等于0.46 OP/B（每字节操作）。正如我们在第5章“存储架构和数据局部性”中讨论的那样，为了使内核的性能接近算术计算资源所支持的水平，这个比率需要更大。我们需要使用类似于第7章“卷积”中讨论的切片技术来提高浮点数到全局内存访问比率。

8.3 共享内存平铺的stencil扫描

正如我们在第5章“内存架构和数据局部性”中所看到的，浮点操作与全局内存访问操作的比率可以通过共享内存平铺显著提高。读者可能怀疑，共享内存平铺的设计对于stencil和卷积几乎是相同的。然而，有一些微妙但重要的区别。

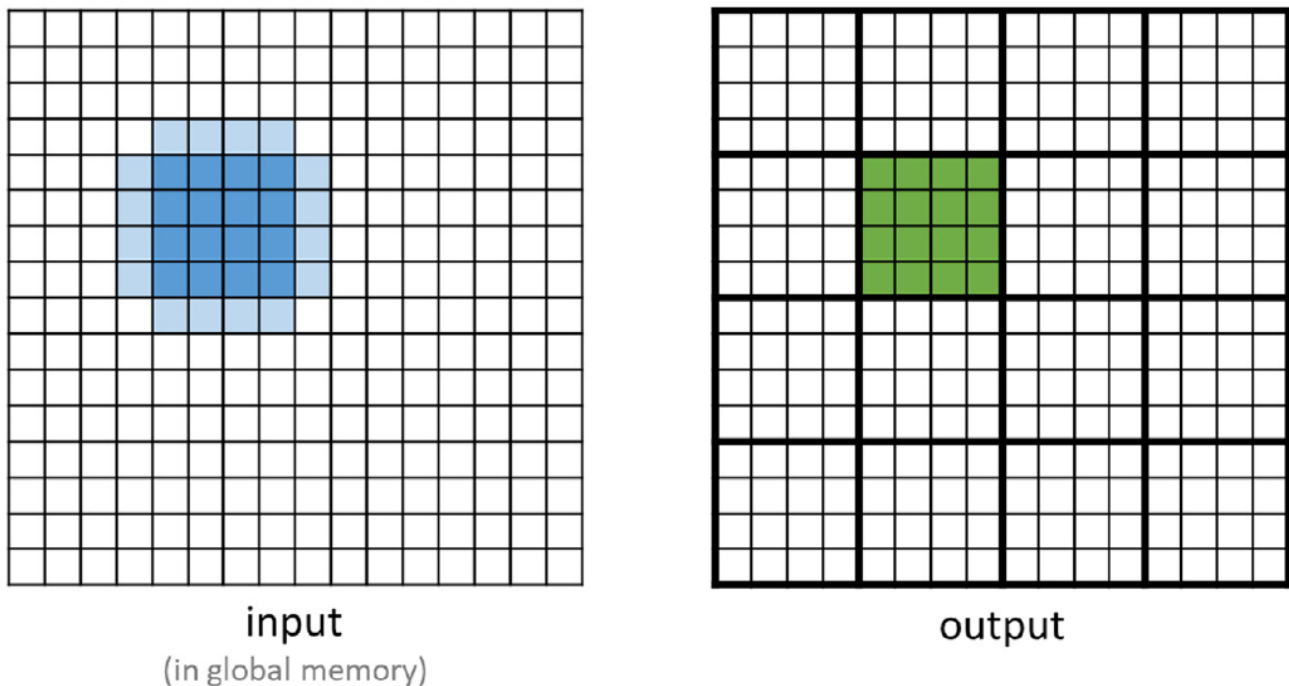


图8.7 2D 五点stencil的输入和输出块。

图8.7显示了应用于小型网格示例的2D 五点stencil的输入和输出块。与图7.11进行快速比较，可以看到卷积和stencil扫描之间的小区别：五点stencil的输入块不包括角落的网格点。当我们在本章后面探讨寄存器平铺时，这个属性将变得重要。对于共享内存平铺，我们可以预期2D 五点stencil中的输入数据重用会比3 x 3卷积要低得多。正如我们在第7章“卷积”中讨论的那样，2D 3 x 3卷积的算术与全局内存访问比率的上限是4.5 OP/B。然而，对于2D 五点stencil，这个比率的上限只有2.5 OP/B。这是因为每个输出网格点值仅使用五个输入网格值，而在3x3x3卷积中是九个输入像素值。

当维度和stencil的阶数增加时，差异会更加显著。例如，如果我们将2D stencil的阶数从1（每边一个网格点，五点stencil）增加到2（每边两个网格点，九点stencil），那么比率的上限是4.5 OP/B，而其对应的2D 5 x 5卷积的比率上限是12.5 OP/B。当2D stencil的阶数增加到3（每边三个网格点，13点stencil）时，这种差异进一步显著。比率的上限是6.5 OP/B，而其对应的2D 7 x 7卷积的比率上限是24.5 OP/B。

当我们进入3D时，stencil扫描和卷积的算术与全局内存访问比率的差异更加显著。例如，一个3D三阶stencil（每边三个网格点，19点stencil）的上限是9.5 OP/B，而其对应的3D 7 x 7 x 7卷积的上限是171.5 OP/B。也就是说，将输入网格点值加载到共享内存中以进行stencil扫描的好处可能远低于卷积，特别是对于3D，这是stencil的主要用例。正如我们在本章后面将看到的，这种小但显著的差异激励了在第三维度中使用线程合并和寄存器平铺。


```

01 __global__ void stencil_kernel(float* in, float* out, unsigned int N) {
02     int i = blockIdx.z*OUT_TILE_DIM + threadIdx.z - 1;
03     int j = blockIdx.y*OUT_TILE_DIM + threadIdx.y - 1;
04     int k = blockIdx.x*OUT_TILE_DIM + threadIdx.x - 1;
05     __shared__ float in_s[IN_TILE_DIM][IN_TILE_DIM][IN_TILE_DIM];
06     if(i >= 0 && i < N && j >= 0 && j < N && k >= 0 && k < N) {
07         in_s[threadIdx.z][threadIdx.y][threadIdx.x] = in[i*N*N + j*N + k];
08     }
09     __syncthreads();
10     if(i >= 1 && i < N-1 && j >= 1 && j < N-1 && k >= 1 && k < N-1) {
11         if(threadIdx.z >= 1 && threadIdx.z < IN_TILE_DIM-1 && threadIdx.y >= 1
12             && threadIdx.y < IN_TILE_DIM-1 && threadIdx.x >= 1 && threadIdx.x < IN_TILE_DIM-1) {
13             out[i*N*N + j*N + k] = c0*in_s[threadIdx.z][threadIdx.y][threadIdx.x]
14                                     + c1*in_s[threadIdx.z][threadIdx.y][threadIdx.x-1]
15                                     + c2*in_s[threadIdx.z][threadIdx.y][threadIdx.x+1]
16                                     + c3*in_s[threadIdx.z][threadIdx.y-1][threadIdx.x]
17                                     + c4*in_s[threadIdx.z][threadIdx.y+1][threadIdx.x]
18                                     + c5*in_s[threadIdx.z-1][threadIdx.y][threadIdx.x]
19                                     + c6*in_s[threadIdx.z+1][threadIdx.y][threadIdx.x];
20         }
21     }
22 }

```

图8.8 一个使用共享内存切片的三维七点stencil扫描kernel。

由于所有用于加载卷积输入块的策略都直接适用于stencil扫描，因此在图8.8中展示了一个与图7.12中的卷积核类似的核心，其中块的大小与输入块相同，并且在计算输出网格点值时关闭了一些线程。这个核心是从图8.6中的基本stencil扫描核心改编的，因此我们只关注在改编过程中所做的改变。与平铺的卷积核心一样，平铺的stencil扫描核心首先计算了每个线程使用的输入块的起始x、y和z坐标。在每个表达式中减去的值1是因为该核心假定一个3D七点stencil，每边有一个网格点（行02-04）。一般来说，减去的值应该是stencil的阶数。

核心在共享内存中分配一个in_s数组来保存每个块的输入块（行05）。每个线程加载一个输入元素。与平铺的卷积核心一样，每个线程加载包含stencil网格点模式的立方输入块的起始元素。由于在行02-04中进行了减法，所以可能会有一些线程尝试加载网格的ghost cells。条件*i* >= 0, *j* >= 0, and *k* >= 0（行06）防止了这些越界访问。因为块比输出块要大，所以也有可能块的x、y和z维度的末端有一些线程试图访问超出每个维度的网格数组的边界的ghost cells。条件*i* < N, *j* < N, *k* < N（行06）防止了这些越界访问。线程块中的所有线程共同将输入块加载到共享内存中（行07），并使用屏障同步等待，直到所有输入块的网格点都在共享内存中（行09）。

每个块在行10-21中计算其输出块。行10中的条件反映了简化的假设，即输入网格和输出网格的边界点包含初始条件值，并且不需要由核心在迭代中计算。因此，那些输出网格点位于这些边界位置的线程被关闭。请注意，边界网格点在网格的表面形成一层。

在行12-13中的条件关闭了额外的线程，这些线程只是为了加载输入块网格点而启动。这些条件允许那些*i*、*j*和*k*索引值位于输出块内的线程计算由这些索引选择的输出网格点。最后，每个活动线程使用由七点stencil指定的输入网格点计算其输出网格点值。

我们可以通过计算这个核心达到的算术与全局内存访问比来评估共享内存切片的有效性。回想一下，原始的stencil扫描核心实现了0.46 OP/B的比率。使用共享内存切片核心，我们假设每个输入切片是一个立方体，每个维度有*T*个网格点，每个输出切片在每个维度上有*T* - 2个网格点。因此，每个块有(*T* - 2)³个活动线程计算输出网格点值，每个活动线程执行13次浮点乘法或加法操作，总共是13 × (*T* - 2)³次浮点运算。此外，每个块通过执行*T*个每个4字节的加载来加载一个输入切片。因此，切片核心的浮点到全局内存访问比率可以计算如下：

$$\frac{13*(T-2)^3}{4*T^3} = \frac{13}{4} * \left(1 - \frac{2}{T}\right)^3 \quad OP/B$$

即， T 值越大，输入网格点值的重用就越多。当 T 增加时，比率的上限渐近地增加到 $13/4$ ，即 3.25 OP/B 。不幸的是，当前硬件块大小的 1024 限制使得很难获得较大的 T 值。 T 的实际限制是 8 ，因为一个 $8 \times 8 \times 8$ 的线程块有总共 512 个线程。此外，用于输入切片的共享内存量与 T^3 成正比。因此，较大的 T 会大幅增加共享内存的消耗。这些硬件约束迫使我们使用小的切片大小。相比之下，卷积通常用于处理 $2D$ 图像，可以使用较大的切片尺寸（例如 32×32 ）。

硬件对 T 值的小限制有两个主要缺点。第一个缺点是它限制了重用比率，因此限制了计算到内存访问比率。对于 $T=8$ ，七点stencil的比率仅为 1.37 OP/B ，远远低于 3.25 OP/B 的上限。随着 T 值减小，重用比率会下降，这是由于halo开销造成的。如我们在第7章中讨论的那样，halo元素的重用性低于非halo元素的重用性。随着输入切片中halo元素的比例增加，数据重用比率和浮点到全局内存访问比率都会降低。例如，对于半径为1的卷积滤波器，一个 32×32 的 $2D$ 输入切片有 1024 个输入元素。相应的输出切片有 $30 \times 30 = 900$ 个元素，这意味着 $1024 - 900 = 124$ 个输入元素是halo元素。输入切片中halo元素的比例约为 12% 。相比之下，对于一阶三维stencil，一个 $8 \times 8 \times 8$ 的 $3D$ 输入切片有 512 个元素。相应的输出切片有 $6 \times 6 \times 6 = 216$ 个元素，这意味着 $512 - 216 = 296$ 个输入元素是halo元素。输入切片中halo元素的比例约为 58% ！

小切片尺寸的第二个缺点是对内存合并有不利影响。对于 $8 \times 8 \times 8$ 的切片，每个包含 32 个线程的warp将负责加载四行不同的切片，每行有八个输入值。因此，在同一次加载指令中，warp的线程将访问至少四个远离的全局内存位置。这些访问无法合并，将导致DRAM带宽被低效利用。因此， T 需要一个更大的值，使得重用水平接近 3.25 OP/B ，并实现对DRAM带宽的充分利用。对于 T 值的更大需求，激发了我们将在接下来的部分中介绍的方法。

8.4 线程粗化

正如我们在前一节中提到的，stencil通常应用于 $3D$ 网格，并且stencil模式的稀疏特性可能使stencil扫描比卷积更难以利用共享内存切片。本节介绍了使用线程粗化来克服块大小限制的方法，通过将每个线程的工作从计算一个网格点值粗化到计算一系列网格点值，如图8.9所示。回想一下，在第6.3节中，线程粗化将并行工作单元部分串行化到每个线程中，并减少了并行性所付出的代价。在这种情况下，所付出的并行性代价是由于每个块加载halo元素导致的低数据重用。

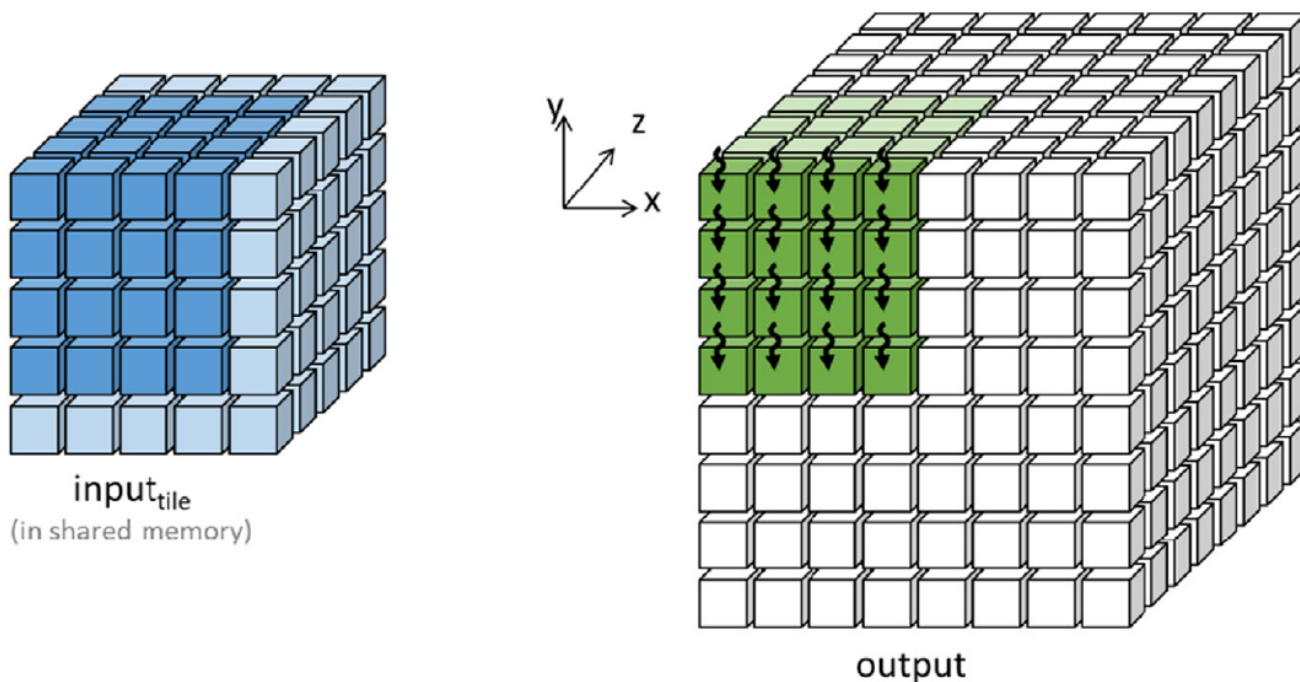


图8.9 z 方向上的线程粗化，用于 $3D$ 七点stencil扫描。

在图8.9中，我们假设每个输入切片由 $T^3=6^3=216$ 个网格点组成。请注意，为了使输入切片的内部可见，我们剥离了切片的前部、左部和顶部层。我们还假设每个输出切片由 $(T-2)^3=4^3=64$ 个网格点组成。图中的坐标系显示了输入和输出的x、y和z方向。每个输入切片的x-y平面由 $6^2=36$ 个网格点组成，而每个输出切片的x-y平面由 $4^2=16$ 个网格点组成。分配给处理此切片的线程块与一个输入切片的x-y平面中的线程数相同（即 6×6 ）。在图中，我们仅显示了计算输出切片值的线程块的内部线程（即 4×4 ）。

```

01 __global__ void stencil_kernel(float* in, float* out, unsigned int N) {
02     int iStart = blockIdx.z*OUT_TILE_DIM;
03     int j = blockIdx.y*OUT_TILE_DIM + threadIdx.y - 1;
04     int k = blockIdx.x*OUT_TILE_DIM + threadIdx.x - 1;
05     __shared__ float inPrev_s[IN_TILE_DIM][IN_TILE_DIM];
06     __shared__ float inCurr_s[IN_TILE_DIM][IN_TILE_DIM];
07     __shared__ float inNext_s[IN_TILE_DIM][IN_TILE_DIM];
08     if(iStart-1 >= 0 && iStart-1 < N && j >= 0 && j < N && k >= 0 && k < N) {
09         inPrev_s[threadIdx.y][threadIdx.x] = in[(iStart - 1)*N*N + j*N + k];
10     }
11     if(iStart >= 0 && iStart < N && j >= 0 && j < N && k >= 0 && k < N) {
12         inCurr_s[threadIdx.y][threadIdx.x] = in[iStart*N*N + j*N + k];
13     }
14     for(int i = iStart; i < iStart + OUT_TILE_DIM; ++i) {
15         if(i + 1 >= 0 && i + 1 < N && j >= 0 && j < N && k >= 0 && k < N) {
16             inNext_s[threadIdx.y][threadIdx.x] = in[(i + 1)*N*N + j*N + k];
17         }
18         __syncthreads();
19         if(i >= 1 && i < N - 1 && j >= 1 && j < N - 1 && k >= 1 && k < N - 1) {
20             if(threadIdx.y >= 1 && threadIdx.y < IN_TILE_DIM - 1
21                && threadIdx.x >= 1 && threadIdx.x < IN_TILE_DIM - 1) {
22                 out[i*N*N + j*N + k] = c0*inCurr_s[threadIdx.y][threadIdx.x]
23                                     + c1*inCurr_s[threadIdx.y][threadIdx.x-1]
24                                     + c2*inCurr_s[threadIdx.y][threadIdx.x+1]
25                                     + c3*inCurr_s[threadIdx.y+1][threadIdx.x]
26                                     + c4*inCurr_s[threadIdx.y-1][threadIdx.x]
27                                     + c5*inPrev_s[threadIdx.y][threadIdx.x]
28                                     + c6*inNext_s[threadIdx.y][threadIdx.x];
29             }
30         }
31         __syncthreads();
32         inPrev_s[threadIdx.y][threadIdx.x] = inCurr_s[threadIdx.y][threadIdx.x];
33         inCurr_s[threadIdx.y][threadIdx.x] = inNext_s[threadIdx.y][threadIdx.x];
34     }
35 }

```

图8.10 z方向上的线程粗化的内核，用于3D七点stencil扫描。

图8.10显示了在3D七点stencil扫描中沿着z方向进行线程粗化的核心。其思想是线程块在z方向（向内）进行迭代，计算每次迭代中输出切片的一个x-y平面中网格点的值。核心首先将每个线程分配给输出的一个x-y平面中的一个网格点（行03~04）。请注意，i是每个线程计算的输出切片网格点的z索引。在每次迭代中，块中的所有线程都将处理一个输出切片的x-y平面；因此，它们将计算具有相同z索引的输出网格点值。

每个块最初需要将包含计算最靠近读者（用线程标记）的输出瓦片平面的所有点所需的三个输入瓦片平面加载到共享内存中。这通过使块中的所有线程加载第一层（行08至10）到共享内存数组inPrev_s，并将第二层（行11至13）加载到共享内存数组inCurr_s中来实现。对于第一层，inPrev_s从已被剥离以便内部层可见性的输入瓦片的前层加载而来。

在第一次迭代中，块中的所有线程合作加载当前输出瓦片层所需的第三层到共享内存数组inNext_s（行15至17）。然后，所有线程等待在屏障上（行18），直到所有线程都完成了输入瓦片层的加载。行19至21中的条件具有与图8.8中的共享内存内核中其对应部分相同的作用。

然后，每个线程使用inCurr_s中存储的四个x-y邻居、inPrev_s中的z邻居和inNext_s中的z邻居计算当前输出瓦片平面的输出网格点值。所有线程在屏障处等待，以确保每个线程都在继续下一个输出瓦片平面之前完成了计算。一旦通过屏障，所有线程合作将inCurr_s的内容移动到inPrev_s，并将inNext_s的内容移动到inCurr_s。这是因为当线程在z方向移动一个输出平面时，输入瓦片平面的角色发生变化。因此，通过每个迭代结束时，块具有计算下一次迭代的输出瓦片平面所需的三个输入瓦片平面中的两个。然后，所有线程进入下一个迭代并加载所需输出平面的输入瓦片的第三个平面。为计算下一个输出平面的准备的inPrev_s、inCurr_s和inNext_s的更新映射在图8.11中进行了说明。

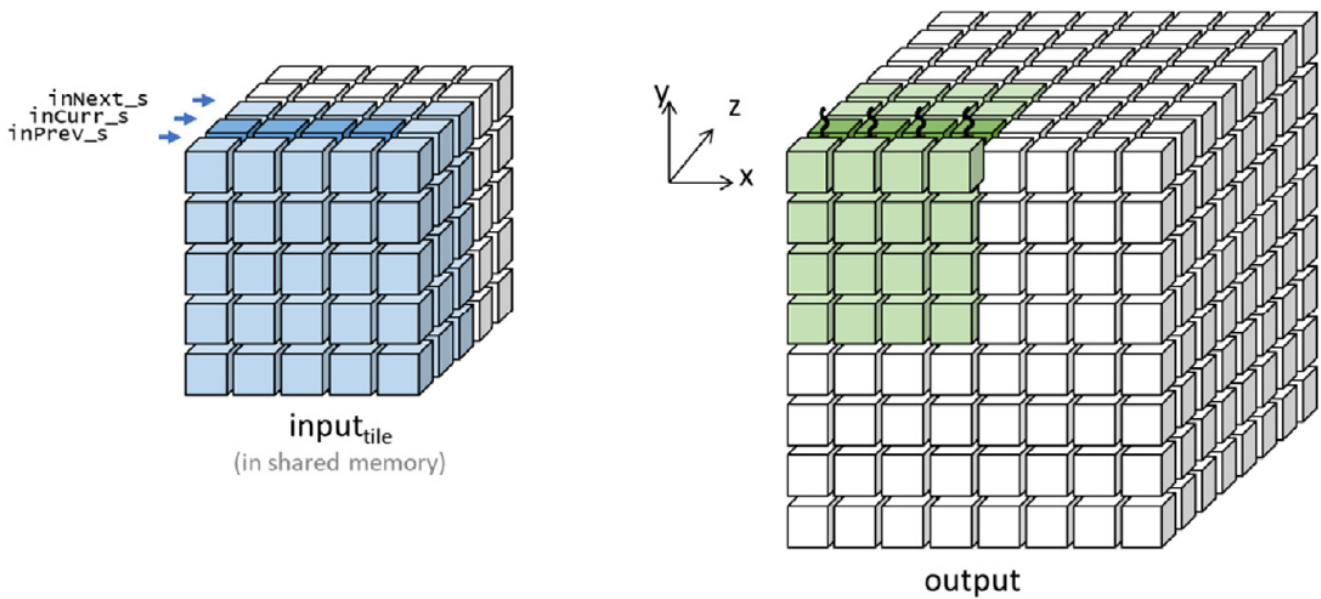


图8.11 第一次迭代后共享内存数组到输入瓦片的映射。

线程粗化内核的优势在于它增加了瓦片的大小而不增加线程数，并且不需要所有输入瓦片的平面都存在于共享内存中。线程块大小现在只有T的平方而不是T的立方，因此我们可以使用更大的T值，例如32，这将导致块大小为1024个线程。使用这个T值，我们可以预期浮点算术到全局内存访问比率将为 $\frac{13}{4} \times (1 - \frac{2}{24})^2 = 2.68 \text{ OP/B}$ ，这比原始共享内存平铺内核的1.37 OP/B比率有了显著的改善，并且更接近3.25 OP/B的上限。此外，在任何时候，共享内存中只需要有三个输入瓦片的层。现在，共享内存的容量要求是3T的平方元素，而不是T的立方元素。对于T为32，共享内存消耗现在处于合理水平，为 $3 \times 32 \times 32 \times 4B = 12KB$ 每个块。

8.5 寄存器瓦片化

某些图案模式的特殊特征可以引发新的优化机会。在这里，我们介绍一种针对只涉及中心点沿x、y和z方向的邻居的图案特别有效的优化。图8.3中的所有图案均符合此描述。图8.10中的3D七点图案扫描核体现了这一特性。每个inPrev_s和inNext_s元素仅由计算具有相同x-y索引的输出瓦片网格点的一个线程使用。只有inCurr_s元素被多个线程访问，确实需要放在共享内存中。inPrev_s和inNext_s中的z邻居可以留在单个用户线程的寄存器中。

我们利用这一特性来设计图8.12中的寄存器瓦片化核。该核是基于图8.10中线程合并核进行了一些简单但重要的修改。我们将重点关注这些修改。首先，我们创建了三个寄存器变量inPrev、inCurr和inNext（第05、07、08行）。寄存器变量inPrev和inNext替换了共享内存数组inPrev_s和inNext_s。相比之下，我们保留了

inCurr_s，以允许 x-y 邻居网格点值在线程之间共享。因此，该核使用的共享内存量减少到图 8.12 中核的三分之一。

```

01 __global__ void stencil_kernel(float* in, float* out, unsigned int N) {
02     int iStart = blockIdx.z*OUT_TILE_DIM;
03     int j = blockIdx.y*OUT_TILE_DIM + threadIdx.y - 1;
04     int k = blockIdx.x*OUT_TILE_DIM + threadIdx.x - 1;
05     float inPrev;
06     __shared__ float inCurr_s[IN_TILE_DIM][IN_TILE_DIM];
07     float inCurr;
08     float inNext;
09     if(iStart-1 >= 0 && iStart-1 < N && j >= 0 && j < N && k >= 0 && k < N) {
10         inPrev = in[(iStart - 1)*N*N + j*N + k];
11     }
12     if(iStart >= 0 && iStart < N && j >= 0 && j < N && k >= 0 && k < N) {
13         inCurr = in[iStart*N*N + j*N + k];
14         inCurr_s[threadIdx.y][threadIdx.x] = inCurr;
15     }
16     for(int i = iStart; i < iStart + OUT_TILE_DIM; ++i) {
17         if(i + 1 >= 0 && i + 1 < N && j >= 0 && j < N && k >= 0 && k < N) {
18             inNext = in[(i + 1)*N*N + j*N + k];
19         }
20         __syncthreads();
21         if(i >= 1 && i < N - 1 && j >= 1 && j < N - 1 && k >= 1 && k < N - 1) {
22             if(threadIdx.y >= 1 && threadIdx.y < IN_TILE_DIM - 1
23                 && threadIdx.x >= 1 && threadIdx.x < IN_TILE_DIM - 1) {
24                 out[i*N*N + j*N + k] = c0*inCurr
25                                     + c1*inCurr_s[threadIdx.y][threadIdx.x-1]
26                                     + c2*inCurr_s[threadIdx.y][threadIdx.x+1]
27                                     + c3*inCurr_s[threadIdx.y+1][threadIdx.x]
28                                     + c4*inCurr_s[threadIdx.y-1][threadIdx.x]
29                                     + c5*inPrev
30                                     + c6*inNext;
31             }
32         }
33         __syncthreads();
34         inPrev = inCurr;
35         inCurr = inNext;
34         inCurr_s[threadIdx.y][threadIdx.x] = inNext_s;
35     }
35 }

```

改为inNext



图8.12 对于3D七点图案扫描，具有线程合并和沿z方向的寄存器瓦片化的核。

之前加载前一个和当前输入瓦片平面（第 09-15 行）以及在每次新迭代之前加载输入瓦片的下一个平面（第 17-19 行）均使用寄存器变量作为目标进行。因此，输入瓦片“活动部分”的三个平面在同一块的线程之间保持在寄存器中。此外，核始终在共享内存中维护输入瓦片的当前平面的副本（第 14 和 34 行）。也就是说，输入瓦片平面的 x-y 邻居始终对需要访问这些邻居的所有线程可用。

图 8.10 和图 8.12 中的核均将输入瓦片的活动部分保留在芯片内存中。活动部分的平面数量取决于图案的顺序，对于 3D 七点图案，该数量为 3。图 8.12 中的线程合并和寄存器瓦片化核相比图 8.10 中的线程合并核具有两个优点。首先，现在许多对共享内存的读取和写入已转移到寄存器中。由于寄存器具有比共享内存更低的延迟和更高的带宽，我们预计代码运行速度会更快。其次，每个块只消耗共享内存的三分之一。当然，这是以每个线程使用三个额外寄存器为代价的，或者假设每个块为 32x32 块，则每个块多使用 3072 个寄存器。读者应

该记住，对于高阶图案，寄存器使用量会更高。如果寄存器使用量成为问题，可以考虑将一些平面存储在共享内存中。这种情况代表了通常需要在共享内存和寄存器使用之间进行的常见权衡。

总体而言，数据重用现在分布在寄存器和共享内存中。全局内存访问次数没有改变。如果考虑到寄存器和共享内存，整体数据重用与仅使用共享内存来处理输入瓦片的线程合并核相同。因此，当我们将寄存器瓦片化应用于线程合并时，对全局内存带宽的消耗没有影响。

需要注意的是，将数据瓦片集体存储在块的线程寄存器中的想法并非新鲜事物。在第 3 章、多维网格和数据中，第 5 章、内存架构和数据局部性中的矩阵乘法核以及第 7 章、卷积中的卷积核中，我们将每个线程计算的输出值存储在该线程的寄存器中。因此，由块计算的输出瓦片被集体存储在该块的线程寄存器中。因此，寄存器瓦片化并不是一种新的优化方法，而是我们之前应用过的方法。在这一章节中，它变得更加明显，因为我们现在使用寄存器来存储输入瓦片的一部分：在计算过程中，同一瓦片有时存储在寄存器中，有时存储在共享内存中。

8.6 总结

在本章中，我们深入研究了图案扫描计算，它看起来只是具有特殊滤波器图案的卷积。然而，由于这些图案来自于微分方程求解中的离散化和数值逼近的导数，它们具有两个特点，促使并实现了新的优化。首先，图案扫描通常在 3D 网格上进行，而卷积通常在 2D 图像或 2D 图像的少量时间切片上进行。这使得两者之间的瓦片化考虑有所不同，并促使对 3D 图案进行线程合并，以实现更大的输入瓦片和更多的数据重用。其次，图案扫描有时可以实现输入数据的寄存器瓦片化，进一步提高数据访问吞吐量并缓解共享内存压力。